Temporal Network Analysis Toolkit 1.0.2: User Guide

Release 1.0.2

Toolkit contributors

Oct 11, 2026

CONTENTS

1 Installation and first analysis **2**
- 1.1 Install . . . 2
- 1.2 First network . . . 2
- 1.3 Build and browse . . . 3

2 Inputs and time parameters **4**
- 2.1 Shapes and conventions . . . 4
- 2.2 Window and lag . . . 4
- 2.3 Hops and latency units . . . 5
- 2.4 Lifetime and search limits . . . 5

3 Signals, FC and null models **6**
- 3.1 Methods versus tags . . . 6
- 3.2 Complete signal workflow . . . 6
- 3.3 Thresholding . . . 7
- 3.4 All supported analysis tags . . . 7

4 Choosing and interpreting measures **11**
- 4.1 Distance and integration . . . 11
- 4.2 Return latency and retention . . . 11
- 4.3 Dynamism and memory . . . 12
- 4.4 Clustering, partners and communities . . . 12

5 Output layouts and summaries **13**
- 5.1 Distance matrices . . . 13
- 5.2 Distance vectors . . . 13
- 5.3 Dynamism order . . . 14
- 5.4 Node-score summaries . . . 14
- 5.5 NaN, infinity and padding . . . 14
- 5.6 Aliases . . . 14

6 Parallel computation and troubleshooting **15**
- 6.1 Start small . . . 15
- 6.2 Process-worker example . . . 15
- 6.3 Scratch and checkpointing . . . 16
- 6.4 Common problems . . . 16

7 API reference **17**
- 7.1 Connectivity . . . 17
- 7.2 Preparation . . . 20
- 7.3 Surrogates . . . 21
- 7.4 Distance and return . . . 21
- 7.5 Dynamism and memory . . . 25
- 7.6 Structure . . . 28
- 7.7 Modularity . . . 32
- 7.8 Network generators . . . 35

This guide explains how to build temporal networks, choose measures and interpret outputs with **tnet_analysis 1.0.2**. Start with small examples before scaling to many nodes, frames or subjects.

The workflow is: signals (optional) → connectivity (optional) → binary network frames → measures. Synthetic networks can enter at the network stage. All examples use generated data and need no external dataset.

CHAPTER

ONE

INSTALLATION AND FIRST ANALYSIS

1.1 Install

Use Python 3.9 or newer. From the source directory:

```
python -m pip install .
```

For an editable installation with parallel, documentation and test tools:

```
python -m pip install -e ".[parallel,docs,test]"
```

NumPy, SciPy, scikit-learn and Numba are core dependencies. Joblib and tqdm are optional parallel tools. NetworkX is an optional test reference, not a runtime requirement for modularity.

1.2 First network

The example keeps a zero-diagonal network for structure and a waiting-diagonal copy for propagation and dynamism.

```
"""Analyze a small generated temporal network."""
import numpy as np
from tnet_analysis.latency_measures import (
    ComputeSmartTemporalDistanceMeasures, ComputeDrunkTemporalDistanceMeasures,
    ComputeReturnLatency,
)
from tnet_analysis.dynamism_and_memory import ComputeDynamism, ComputeRetention
from tnet_analysis.segregation_and_cohesion import (
    ComputeSpatialClustering, ComputeTemporalClustering,
)

rng = np.random.default_rng(42)
upper = np.triu(rng.binomial(1, 0.3, size=(30, 8, 8)), k=1)
structure_network = (upper + upper.transpose(0, 2, 1)).astype(np.uint8)
walk_network = structure_network.copy()
walk_network[:, np.arange(8), np.arange(8)] = 1

results = {
    "smart": ComputeSmartTemporalDistanceMeasures(walk_network, summary_stat="both
↪"),
    "drunk": ComputeDrunkTemporalDistanceMeasures(
        walk_network, n_reps=20, random_seed=43, summary_stat="both"),
    "dynamism": ComputeDynamism(walk_network),
    "spatial_clustering": ComputeSpatialClustering(structure_network, "both"),
    "temporal_clustering": ComputeTemporalClustering(
        structure_network, delta=1, summary_stat="both"),
    "return": ComputeReturnLatency(walk_network, summary_stat="both"),
    "retention": ComputeRetention(structure_network, "both", delta=5),
```

(continues on next page)

(continued from previous page)

```
}
for name, values in results.items():
    print(name, values)
```

Download `quick_start.py` and run it:

```
python quick_start.py
```

The first compiled call includes Numba compilation. Subsequent calls are usually faster. The distance vector has eight entries; *Output layouts and summaries* (page 13) explains their order.

1.3 Build and browse

From the source directory:

```
python -m pip install ".[docs]"
python -m sphinx -W --keep-going -b html docs docs/_build/html
```

Open `docs/_build/html/index.html` in a browser. The site includes search and an API reference extracted from library docstrings. For a continuous HTML page, use builder `singlehtml` and a different output directory.

To build the PDF using a TeX installation with XeLaTeX and latexmk:

```
python -m sphinx -b latex docs docs/_build/latex
latexmk -xelatex -interaction=nonstopmode -halt-on-error -cd docs/_build/latex/tnet_
↪user_guide.tex
mkdir -p output/pdf
cp docs/_build/latex/tnet_user_guide.pdf output/pdf/tnet_analysis-1.0.2-user-guide.
↪pdf
```

The PDF has a contents page, page numbers, bookmarks and the API reference. HTML builds do not require TeX. Rebuild the PDF when guide content changes, then rebuild HTML so its downloadable copy is refreshed.

CHAPTER

TWO

INPUTS AND TIME PARAMETERS

2.1 Shapes and conventions

- One subject's signals: `(L, N)` — samples, nodes.
- Connectivity or adjacency: `(T, N, N)` — frames, nodes, nodes.
- One snapshot: `(N, N)`.
- Subject batches: `(subjects, L, N)`; analyze individual subjects unless the runner explicitly supports batching.

Move the time axis before passing a `(nodes, nodes, frames)` tensor. Most measures assume symmetric, undirected frames. Not every function validates every assumption: check finite values, symmetry and binary edge values. Signed FC is not a binary adjacency tensor.

Structure measures should receive zero diagonals. Smart and random walkers force waiting diagonals internally. Give return latency a waiting-diagonal network. Give `ComputeDynamism` diagonal ones too: its edge-count calculation subtracts one self-entry per node per frame. Its other five components use off-diagonal edges.

Average degree counts neighbors, not weighted strength. Modularity and participation support nonnegative weights. Directed modularity scoring requires supplied labels and `directed=True`; automatic Louvain detection is undirected. This does not imply directed support for other measures.

2.2 Window and lag

For Pearson FC, `window` counts signal samples; `lag` is the step between window starts. The dispatch function defaults to 20 and 1. The lower-level Pearson function requires both arguments. Use positive integers, at least two samples per correlation window and a recording longer than the window.

This implementation returns `int((L-window)/lag)` Pearson frames. It does not add an endpoint window: 60 samples, window 20 and lag 5 yield eight frames, not nine. Instantaneous FC yields one frame per signal sample, ignoring window and lag.

Neighborhood memory's lag counts **adjacency frames**. Partner diversity's `window_size` groups adjacency frames, independently of the FC window.

2.3 Hops and latency units

`hops_per_frame=1` is the default. Larger values permit multiple substeps in each frame. Smart propagation explores reachability; a random walk selects one available neighbor, including waiting, at each substep.

For first arrival at substep s with hop allowance h:

- `latency_unit="frames"` returns ceil(s/h).
- `"frame_fraction"` returns s/h.
- `"substeps"` returns s.

Substeps include waiting, not just non-self edge traversals. Fractional-frame timing is a model convention, not measured within-frame physical timing.

```
>>> import numpy as np
>>> from tnet_analysis.latency_measures import SmartWalker
>>> snapshot = np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]])
>>> network = np.repeat(snapshot[None, :, :], 3, axis=0)
>>> SmartWalker(network)[0, 2]
2.0
>>> SmartWalker(network, hops_per_frame=2)[0, 2]
1.0
>>> SmartWalker(network, hops_per_frame=2, latency_unit="substeps")[0, 2]
2.0
```

2.4 Lifetime and search limits

`ComputeTemporalClustering(delta=1)` uses consecutive central connections with closure in the second frame. At larger delta, central connections can be separated by up to delta frames; closure is accepted from the second connection's frame through delta-1 frames later. Each eligible triplet is counted once, even with repeated closing edges. Recording boundaries truncate closure windows without wrapping.

`ComputeRetention(delta=None)` considers all past frames. A positive delta restricts history to max(0,t-delta) through t-1. Current contacts cannot be their own history. The first nonempty observation is excluded. Later nonempty frames can score zero when all earlier contacts expire; empty neighborhoods are skipped.

Return latency's `max_latency` limits the **frames** searched from each start, even with multiple hops. It defaults to the recording horizon. Shortening it changes the estimate, not just runtime.

CHAPTER

THREE

SIGNALS, FC AND NULL MODELS

3.1 Methods versus tags

`fc_method="original1"` selects sliding-window Pearson FC; `"original2"` selects instantaneous z-score products. These are methods, not null-model tags. `tag="original"` means no randomization for either method. Select method and tag independently.

Instantaneous FC uses `FC[t,i,j] = z[t,i]*z[t,j]`, standardizing each region over the full recording. Products are signed and not bounded by [-1,1]. With population standardization (ddof=0), their time average equals whole-record Pearson correlation. Constant regions are rejected.

3.2 Complete signal workflow

```
"""Compare FC methods and transformations using generated signals."""
import numpy as np
from tnet_analysis.analysis_preparation import PrepareConnectivity,␣
↪PrepareTemporalNetwork
from tnet_analysis.dfc_to_tnet import FindThresholdsForAverageDegrees
from tnet_analysis.latency_measures import (
    ComputeAverageDegree, ComputeSmartTemporalDistanceMeasures,
)

rng = np.random.default_rng(100)
signals = rng.standard_normal((100, 8))
degrees = [1.0, 2.0]
tags = [
    "original", "emp_static", "time", "uniform", "phase_rand0", "phase_rand1",
    "er_static", "sw_static", "sf_static", "eAct", "eActTemplate",
    "eRndTemplate", "erand_er", "erand_sws", "erand_sfs", "erand_swr",
    "erand_sfr",
]

for method_index, method in enumerate(["original1", "original2"]):
    for tag_index, tag in enumerate(tags):
        seed = 1000 + 100 * method_index + 10 * tag_index
        fcs = PrepareConnectivity(
            signals, tag=tag, fc_method=method, window=20, lag=5,
            random_seed=seed,
        )
        thresholds = FindThresholdsForAverageDegrees(fcs, degrees)
        for degree_index, degree in enumerate(degrees):
            network = PrepareTemporalNetwork(
                fcs, tag=tag, target_degree=degree,
                threshold=thresholds[degree_index], seed=seed + degree_index,
            )
            achieved, _ = ComputeAverageDegree(network)
```

(continues on next page)

(continued from previous page)

```
            metrics = ComputeSmartTemporalDistanceMeasures(network, summary_stat=
↪"both")
            print(method, tag, degree, round(achieved, 3), metrics)
```

Download `signal_workflow.py`. `PrepareConnectivity` transforms signals before FC. `PrepareTemporalNetwork` thresholds that FC and transforms the network. Passing unrandomized FC with a phase tag into the second function does not retroactively randomize signals.

3.3 Thresholding

Temporal binarization finds one signed threshold for the FC tensor and applies `FC > threshold` to every frame. It targets average degree over the recording, not identical degree in every frame. `FindThresholdsForAverageDegrees` uses a finite candidate grid (resolution 100 by default); requested degree is approximate. Record the achieved degree using `ComputeAverageDegree`.

`emp_static` averages FC over time, selects strongest **absolute-valued** off-diagonal entries and repeats that static network. Ties can exceed the requested edge count. This differs from temporal signed thresholding.

For several degrees, compute FC once, find all thresholds together and pass the corresponding threshold into network preparation. Recompute thresholds when signals or FC method change; phase surrogates need their own FC.

3.4 All supported analysis tags

There are 17 analysis tags. They act at different stages: phase surrogates modify signals before FC, `emp_static` changes binarization, and network nulls transform the thresholded adjacency. None selects an FC method.

Tag	Operation
`original`	Keep the signal and temporal network.
`emp_static`	Repeat a static network formed from mean FC.
`er_static`	Repeat one random snapshot with the mean reference edge count.
`sw_static`	Repeat one small-world-style snapshot with the mean reference edge count.
`sf_static`	Repeat one rank-weighted snapshot with the mean reference edge count.
`time`	Shuffle adjacency frames, not raw signals.
`uniform`	Generate random frames with approximately constant edge count matched to the mean input density.
`eAct`	Randomize each edge's active frames, preserving its activation total.
`eActTemplate`	Use an empirically ranked edge template with framewise rewiring.
`eRndTemplate`	Use a randomly ranked edge template with framewise rewiring.
`erand_er`	Randomize edge locations while retaining each frame's edge count.
`erand_sws`	Generate small-world-style frames with a shared node ordering.
`erand_sfs`	Generate rank-weighted frames with a shared node ordering.
`erand_swr`	Generate small-world-style frames, then independently relabel each frame.
`erand_sfr`	Generate rank-weighted frames, then independently relabel each frame.
`phase_rand0`	Add independent Fourier phases per region.
`phase_rand1`	Add shared Fourier phases across regions.

3.4.1 Reference network

`original` retains the selected signals and temporal thresholding. It is valid with either `fc_method="original1"` or `"original2"`. In network preparation, isolated-node trimming and padding can still change node order; see the warning below.

3.4.2 Static controls

`emp_static` preserves a static representation of mean FC, not the original sequence of framewise edges. All frames are identical. Its absolute-valued edge selection can include strong negative FC; temporal thresholding instead uses signed values. Calling `GenerateNullModel(tnet, "emp_static")` alone does not average FC or construct this static graph: that step belongs to `PrepareTemporalNetwork` / `BinarizeDFCFromConnectivity`.

`er_static`, `sw_static` and `sf_static` generate one snapshot using the rounded mean reference edge count, then repeat it for all T frames. They remove temporal topology changes and reference edge identities. They do not preserve empirical node degrees, clustering or community structure.

- `er_static` samples the requested number of unordered pairs uniformly without replacement: a fixed-edge-count random graph, not G(N,p).
- `sw_static` first selects pairs in increasing node-index separation, then rewires a fraction (0.02 by default). This is a small-world-style local backbone, not an exact Watts-Strogatz ring-lattice implementation.
- `sf_static` samples pairs without replacement with probabilities proportional to the product of node-rank weights, where rank weight is proportional to rank**(-gamma), gamma=2.5 by default. This encourages hubs but does not guarantee a fitted power-law degree distribution.

To change generator-specific parameters, use the corresponding lower-level generator. The preparation wrapper does not expose their rewiring fraction or power-law exponent.

3.4.3 Time-order and uniform-density controls

`time` permutes whole adjacency frames. It preserves each frame's topology, the distribution of framewise edge counts and each edge's total activations. It changes temporal order and usually changes consecutive-frame similarity, time-respecting paths and recurrence intervals. It does not shuffle raw time-series samples or randomize edges within a frame.

`uniform` generates fresh random edge sets with nearly equal counts across frames, matching the mean reference edge count. Integer rounding can produce one-edge differences. Neither the original count sequence nor per-edge activation totals are preserved. This is not a degree-sequence-preserving rewiring or a fixed-probability Bernoulli graph model.

3.4.4 Edge-activation control

`eAct` independently chooses new active-frame positions for each edge, without replacement, retaining exactly its number of active frames. Thus edge identities, total density and the time-aggregated weighted graph are preserved, but per-frame edge counts, simultaneous activations, run lengths and temporal ordering are not. Always-active and never-active edges remain so. It is not the same as `time`, which permutes whole frames.

3.4.5 Ranked-template controls

Both template tags retain the original edge count in each frame. A frame with k edges starts with the first k edges of a common ranked template; floor(p_rewire*k) of those edges are selected for replacement. They model shared topology with controlled variation rather than unconstrained random frames. Selecting an edge for replacement does not guarantee that it changes: removed edges can be selected again.

`eActTemplate` ranks edges by their reference activation totals, highest first. This retains an empirical preference ordering, not exact per-edge activation totals. By default never-active edges are excluded from weighted replacement candidates. `eRndTemplate` instead assigns one random ranking to all possible edges; reference activation preferences are discarded, while the shared template still induces temporal persistence.

The lower-level interface supports:

- `p_rewire=0.02`: fraction selected for replacement per frame; 0 repeats template prefixes (identical graphs when edge counts are identical).
- `alpha_rank=1.0`: replacement preference decays as 1/(rank+1)**alpha_rank; 0 gives uniform rank weights.
- `exclude_never_active=True`: suppresses never-active replacement candidates for `eActTemplate` only. A sparse candidate pool can trigger an unweighted fallback, so this is not an unconditional exclusion rule.
- `eta_readd=1.0`: positive multiplier controlling the preference for re-adding edges selected for removal.

`GenerateNullModel` forwards these options to the template generator; `PrepareTemporalNetwork` uses the defaults. For customized templates:

```
from tnet_analysis.network_generators import GenerateNullModel

template_network, n_core = GenerateNullModel(
    reference_network, "eActTemplate", seed=123,
    p_rewire=0.1, alpha_rank=1.0,
    exclude_never_active=True, eta_readd=0.5,
)
```

Use a binary reference network with diagonal ones. This call operates after FC construction and thresholding; it does not accept raw signals.

3.4.6 Framewise synthetic controls

All five `erand_*` tags regenerate frames with the corresponding reference edge counts, retaining the count sequence but not empirical edge identities.

- `erand_er` samples a uniform fixed-size edge set in each frame.
- `erand_sws` generates small-world-style local edges using the same node ordering across frames. The `s` suffix indicates shared labels.
- `erand_sfs` generates rank-weighted edges using the same node ranks across frames. Shared ranks induce persistent hub preferences.
- `erand_swr` uses the small-world-style construction and independently permutes node labels in each frame afterwards.
- `erand_sfr` does the same for the rank-weighted construction. The `r` suffix indicates randomized labels, not a different edge-count model.

Relabeling retains each generated frame's unlabeled topology but removes fixed node identities across frames. It does not guarantee zero persistence or equal node degrees in a particular realization. The small-world generator currently resets NumPy's RNG when a seed is passed for each frame; equal edge-count frames can therefore repeat the same graph before relabeling. Do not interpret `erand_sws` as independent draws at fixed seed. These tags use the same lower-level small-world and rank-weighted constructions described for the static controls.

3.4.7 Signal-phase surrogates

Both phase modes preserve means, Fourier magnitudes and circular autocovariance per region. Shared phases also preserve cross-spectra and sample covariance; independent phases generally do not. Neither guarantees unchanged sliding-window FC or every time-domain feature.

`phase_rand0` uses independent phase offsets for each region and frequency. It disrupts inter-region phase relationships while retaining each signal's power spectrum. `phase_rand1` uses the same frequencywise offsets for all regions, retaining their relative phases and cross-spectra. Neither mode preserves the original samples or their amplitude distribution in general. Recompute FC and thresholds from the surrogate signals before analysis.

3.4.8 Choosing an interface and recording seeds

`ResolveAnalysisTag` recognizes all 17 tags and rejects unknown values. `GenerateNullModel` accepts the 15 network-level tags only: it does not handle phase surrogates or FC-method selection. Its `scale` argument is reserved and does not change generation. The preparation wrapper skips network generation if thresholding yields no off-diagonal edges.

> **Warning**
>
> Some generators trim isolated nodes, build a core and pad it into the leading node positions. Original node identities are not guaranteed. Use aggregate comparisons unless you verify an explicit node mapping.

Record separate seeds for surrogates, nulls, random walks and community detection. Network preparation seeds NumPy's global RNG for generators using it. Process workers isolate that state; concurrent threads do not.

CHAPTER

FOUR

CHOOSING AND INTERPRETING MEASURES

4.1 Distance and integration

Smart latency is earliest arrival from each source starting at the first frame. It explores all available time-respecting options; it is not a single sampled walk or an average over every possible start time. Smart propagation is deterministic for a fixed network and parameter set; its accepted random_seed argument does not affect the search. Drunk latency samples random walks and first-passage times. More trials reduce Monte Carlo variation but increase cost. A finite horizon can leave pairs unvisited even if another possible path exists.

Distance summaries contain:

- **Impermeability:** distance divided by the source's number of reached targets, aggregated over finite pairs. It combines timing and coverage; it is not just inverse irrigation.
- **Latency:** finite pairwise arrival times only.
- **Resistance:** arrival times with unreachable pairs replaced by a penalty.
- **Inaccessibility:** number of unreachable ordered off-diagonal pairs.
- **Irrigation:** number of reached ordered off-diagonal pairs.

With T frames, the default penalty is T+1 for frames and frame fractions, or h*(T+1) for substeps. Keep penalty conventions consistent across comparisons. A larger penalty directly increases resistance when pairs are unreachable. Latency alone excludes those pairs and can look small in a poorly connected network. Read it alongside irrigation.

For N nodes, ordered-pair counts have denominator N*(N-1), not N*(N-1)/2. Changing network size changes raw counts; report any normalization explicitly.

4.2 Return latency and retention

Return latency searches for first non-waiting returns to the starting node, across eligible start frames. It summarizes each node's mean return latency across successful starts. A waiting self-loop alone is not a successful return. The accompanying count averages successful start-frame returns over included nodes: it is **not a count of distinct routes**. Use `return_full=True` for nodewise means, minima, hit counts and included-node indices.

Retention is the fraction of current neighbors encountered in eligible past frames, averaged over eligible nonempty observations per node. It describes recurring partners, not travel back to a source. Nodes with fewer than two nonempty observations have NaN retention. Its lifetime parameter and return latency's search limit answer different questions.

4.3 Dynamism and memory

`ComputeDynamism` returns six scalars in the order given in *Output layouts and summaries* (page 13):

- **Transition probability:** fraction of possible edges changing state between consecutive frames.
- **Global entropy:** normalized Bernoulli entropy of edge activation probabilities across time. Persistent and never-active edges contribute zero.
- **Cosine similarity:** similarity of consecutive edge-state vectors. Two empty frames have similarity one; an empty/nonempty pair has zero.
- **Net fluidity:** global entropy multiplied by (1-cosine similarity). Similarity and fluidity are related but not generally exact opposites.
- **Mutual information:** mean normalized mutual information of consecutive binary edge-state vectors; this is not mutual information between signals.
- **Mean edge count:** average number of undirected off-diagonal edges.

Edge overlap is consecutive edge-set Jaccard, skipping empty unions. **Edge persistence rate** averages the fraction of source-frame edges surviving into the next frame. Frames receive equal weight; this is not a pooled edge-weighted survival fraction. **Neighborhood memory** is nodewise neighborhood Jaccard at a chosen frame lag. Empty unions are skipped; nodes with no eligible comparisons score zero.

Link burstiness is (gap standard deviation - mean gap)/ (gap standard deviation + mean gap), using population standard deviation. Regular gaps give -1; variable gaps can give positive values. `event_mode="activations"` treats every active frame as an event; `"onsets"` treats a contiguous run as one event. At least three events are needed; other entries and the diagonal are NaN. Short recordings, censoring and few events limit interpretation.

4.4 Clustering, partners and communities

Spatial clustering measures connections among each node's neighbors, then averages each node across frames. Degree below two contributes zero. **Temporal clustering** measures temporally ordered triangle closure, with the delta rules in *Inputs and time parameters* (page 4). It is not spatial clustering applied to an aggregated graph. **Transitivity** is global triangle closure relative to connected triples; it is not mean local clustering. Nonnegative weights define edge presence.

Partner stability defaults to 1-mean recurrence gap/T. All recurrence intervals receive equal weight; nodes without recurring partners score zero. Its duration normalization makes recording lengths important. `normalization="gap"` instead returns reciprocal mean gap per frame, not a next-frame survival probability.

Partner diversity is entropy of partners' active-window coverage. Default `normalization="observed"` measures evenness: two equally covered partners can score one, just as many equally covered partners can. `"available"` divides by log(N-1) to incorporate partner breadth. Zero/one-partner nodes score zero; the final partial window is included.

Node persistence averages occupancy over ever-connected partners. It also returns a count exceeding an iid conditional-binomial null. `confidence` sets the upper-tail threshold; `null_probability` supplies p, otherwise density estimates p. `return_details=True` adds scores and tail probabilities. This null assumes independent, homogeneous links and independent frames. Overlapping FC windows, autocorrelation and estimated p do not yield calibrated significance. There is no automatic multiple-testing correction.

Participation is 1-sum of squared community-strength fractions. It measures spread of connections across the supplied partition; isolates have NaN scores. **Modularity** scores within-community connectivity relative to a strength-based null, with resolution parameter gamma. Gamma defaults to 1; increasing it generally favors finer partitions, although a heuristic search need not change monotonically on every run. Louvain is heuristic: seeds and restarts matter. Temporal modularity is the mean of independent snapshot modularities, not a multilayer model with inter-frame community coupling.

CHAPTER

FIVE

OUTPUT LAYOUTS AND SUMMARIES

5.1 Distance matrices

`SmartWalker` returns an (N,N) matrix: NaN on the diagonal, infinity for unreachable pairs, and finite arrival times elsewhere.

`RandomWalker` returns (matrix, trial-averaged metrics), or additionally an (trials,N,N) tensor when `return_fpt_matrix=True`. Its matrix averages only successful first passages; a pair is infinite if no trial reaches it. The metrics summarize each trial first, then average trials. They need not equal a summary recomputed from the returned mean matrix. Likewise, a drunk median entry is the average of trial-specific medians, not the median of pooled first-passage observations.

5.2 Distance vectors

Smart/drunk measure functions default to `summary_stat="mean"`:

1. impermeability
2. latency
3. resistance
4. inaccessibility
5. irrigation

With `"both"`, the eight entries are:

1. impermeability mean
2. impermeability median
3. latency mean
4. latency median
5. resistance mean
6. resistance median
7. inaccessibility
8. irrigation

Counts occur once. Drunk counts can be fractional because they average trials. The impermeability median uses finite source-normalized pair entries. Do not interpret every median as a median over nodes.

5.3 Dynamism order

`ComputeDynamism` returns transition probability, global entropy, cosine similarity, net fluidity, mutual information and mean edge count. These are six scalars, not six node-score distributions.

5.4 Node-score summaries

For functions supporting all summary options:

- `None` or `"percentiles"`: [p05,p25,p50,p75,p95].
- `"mean"` or `"median"`: one scalar.
- `"both"`: [mean,median].

Here the median of node scores is not necessarily the median of all underlying frames or events. Return latency summarizes nodewise mean latencies. Node persistence adds its exceedance count to the summary. Return latency returns (summary,mean successful-return count), unless nodewise details are requested. Burstiness returns a matrix, not this summary schema. Modularity and participation have their own outputs; consult the API rather than assuming one universal shape.

5.5 NaN, infinity and padding

NaN often represents an undefined score or excluded diagonal. Infinity in distance matrices represents non-arrival within the horizon. Zero is a valid measure value, not universally missing data.

Use NaN-aware population summaries only after recording missing-data counts. Do not silently turn infinity into zero. Resistance already handles non-arrivals via its explicit penalty.

The example `funcIntSeg` wrapper has seven metric groups and pads every vector to width eight. Their meaningful widths are [8,8,6,2,2,3,2]. Padding zeros are not observations. Use `METRIC_SPECS` output names to identify meaningful entries, and preserve those names with saved arrays. Package functions themselves do not add wrapper padding.

Save node order, subject IDs, achieved degrees, tags, FC method, time settings, seeds, units and trial counts alongside results.

5.6 Aliases

`ComputeReturnability` aliases `ComputeRetention`; `ComputeCirculationLatency` aliases `ComputeReturnLatency`; `ComputeStaticClustering` aliases `ComputeSpatialClustering`; `LinkBurstiness` aliases `ComputeLinkBurstiness`. Each pair refers to the same implementation. The canonical names are used throughout the guide.

CHAPTER

SIX

PARALLEL COMPUTATION AND TROUBLESHOOTING

6.1 Start small

Run one subject, a few degrees and a modest trial count first. Check shapes, achieved degrees and NaN fractions, then time sparse and dense cases. Increase worker counts gradually, observing memory and scratch use. Hardware threads are not equivalent to physical cores, and more workers do not guarantee a faster job.

A dense adjacency tensor needs T*N*N*itemsize bytes. A 296-frame, 400-node uint8 tensor needs about 45 MiB; float64 needs about 361 MiB. Worker copies, FC tensors, temporary arrays and walk trials add to this. Random-walk first-hit storage scales with trials*N*N even when full FPT output is disabled. Avoid requesting full per-trial matrices unnecessarily. Its dense neighbor table also uses int32 storage proportional to T*N*N. The dynamism implementation forms a T-by-T similarity matrix, so long recordings can add time-axis memory costs as well.

Prepare FC once per subject/method/window/lag and threshold sets once per FC tensor. Reuse each network across metrics. Compute hop-independent metrics once rather than repeating them for every hop allowance. One-hop return uses packed reachability; the multi-hop return branch uses dense matrix propagation, so larger hop allowances can be expensive.

6.2 Process-worker example

```
"""Process subjects in independent workers; no files are written."""
import numpy as np
from joblib import Parallel, delayed, parallel_config
from tnet_analysis.analysis_preparation import PrepareConnectivity,␣
↪PrepareTemporalNetwork
from tnet_analysis.latency_measures import ComputeSmartTemporalDistanceMeasures
from tnet_analysis.segregation_and_cohesion import ComputeSpatialClustering


def ComputeSubject(subject_index, signals):
    seed = 2000 + subject_index
    fcs = PrepareConnectivity(signals, window=20, lag=5)
    network = PrepareTemporalNetwork(fcs, "original", target_degree=2, seed=seed)
    no_diagonal = network.copy()
    diagonal = np.arange(network.shape[1])
    no_diagonal[:, diagonal, diagonal] = 0
    return {
        "subject_index": subject_index,
        "smart": ComputeSmartTemporalDistanceMeasures(network, summary_stat="both"),
        "spatial_clustering": ComputeSpatialClustering(no_diagonal, "both"),
    }


if __name__ == "__main__":
    subjects = np.random.default_rng(200).standard_normal((4, 100, 8))
    with parallel_config(backend="loky", n_jobs=2, inner_max_num_threads=1):
```

(continues on next page)

(continued from previous page)

```
        results = Parallel()(
            delayed(ComputeSubject)(index, signals)
            for index, signals in enumerate(subjects)
        )
    for result in results:
        print(result)
```

Download `parallel_subjects.py`. The worker analyzes one small subject and returns named results. The main guard supports process-based execution. Run it as a script, not by starting workers during module import.

Limit nested numerical-library threads when using process workers. Joblib's Loky configuration below limits supported inner thread pools. Benchmark on the target machine; no fixed worker count is optimal everywhere.

6.3 Scratch and checkpointing

For large shared arrays, Joblib can use disk-backed memory maps. Choose an explicitly configured local scratch directory with enough space, pass it via `temp_folder`, and monitor the filesystem containing that directory, not just the output disk.

Use a scoped temporary directory for one subject/chunk so normal completion cleans it. An abrupt kill can leave files: only remove directories known to belong to stopped runs. Never clear a shared scratch root indiscriminately.

Save completed subjects individually, preferably by writing a temporary file and atomically renaming it. Resume only when the saved shape and configuration match. Neither metric functions nor this simple example automatically implement checkpointing.

6.4 Common problems

- **Non-finite Pearson FC:** check missing samples and constant regions within each window. Decide a documented exclusion or imputation policy; do not blindly replace undefined correlations with zero.
- **No space left on device / pickling error:** check Joblib scratch, available space and inode counts. The pickling message can wrap a disk-write failure.
- **First call is slow:** Numba compilation is expected; compare warmed timings. Set `NUMBA_CACHE_DIR` to a writable folder if needed.
- **Means and medians differ:** sparse node-score distributions can have positive means and zero medians. Inspect scores and eligible counts.
- **Counts look negative:** give dynamism waiting diagonals; do not change node axes or include wrapper padding in calculations.
- **Memory pressure:** lower worker count, use compact binary dtypes, avoid full outputs and keep subject processing bounded.

CHAPTER

SEVEN

API REFERENCE

The signatures and descriptions below come from package docstrings. The guide examples deliberately import library modules, not runner scripts that load external data when executed.

7.1 Connectivity

ComputeDynamicFunctionalConnectivity(
time_series,
window,
lag,
)

Compute dynamic functional connectivity (DFC) matrices from time series.

Parameters

- **time_series** – np.ndarray, shape (T, N). Input time series with T time points and N regions/nodes.
- **window** – int. Length of the sliding time window.
- **lag** – int. Step size (lag) between successive windows.

Returns

fcs – np.ndarray, shape (n_snapshots, N, N). Sequence of functional connectivity matrices computed as Pearson correlation coefficients within each window.

ComputeInstantaneousFunctionalConnectivity(
time_series,
ddof=0,
)

Compute pointwise connectivity as products of region-wise z-scored signals.

Parameters

- **time_series** – np.ndarray, shape (T, N). Finite, real-valued signals with T time points and N regions/nodes. Each region is standardized over time.
- **ddof** – {0, 1}, optional (default=0). Standard-deviation convention: population (0), or sample (1, as in MATLAB's default zscore).

Returns

fcs –

np.ndarray, shape (T, N, N). One frame per input time point,
with fcs[t, i, j] = z[t, i] * z[t, j]. No window or lag is used.

Notes

- With ddof=0, the time-average equals whole-record Pearson correlation. With ddof=1, multiply the time-average by T / (T - 1) to obtain it.
- Products are signed and not bounded by [-1, 1]. Diagonals are z[t, i]**2, not forced to one; binarization handles self-edges separately.
- Constant regions are rejected because their z-scores are undefined.

ComputeFunctionalConnectivity(
time_series,
window=20,
lag=1,
method='original1',
ddof=0,
)

Select sliding-window Pearson or instantaneous z-score-product connectivity.

Parameters

- **time_series** – np.ndarray, shape (T, N). Input time series.
- **window** – int, optional (default=20). Pearson sliding-window length; ignored for 'original2'.
- **lag** – int, optional (default=1). Pearson window step; ignored for 'original2'.
- **method** – {'original1', 'original2'}, optional (default='original1'). 'original1' uses Pearson correlations; 'original2' uses z-score products. The method name 'original' is also accepted as an alias for 'original1'.
- **ddof** – {0, 1}, optional (default=0). Z-score standardization convention; used only for 'original2'.

Returns

fcs –

np.ndarray. Shape (n_snapshots, N, N) for Pearson, or (T, N, N)
for instantaneous products.

FindThresholdsForAverageDegrees(
adj_tensor,
target_degrees,
resolution=100,
)

Find thresholds for several average degrees using one sorted connectivity tensor.

Parameters

- **adj_tensor** – np.ndarray, shape (T, N, N). Temporal connectivity matrices.
- **target_degrees** – array-like. Desired average degrees.
- **resolution** – int, optional (default=100). Number of threshold candidates.

Returns
thresholds – np.ndarray. Selected thresholds in the target-degree order.

FindThresholdForAverageDegree(
adj_tensor,
target_degree,
resolution=100,
)

Find the correlation threshold that produces a desired average degree.

Parameters

- **adj_tensor** – np.ndarray, shape (T, N, N). Temporal connectivity matrices (e.g., correlations or z-score products). Symmetric per snapshot.
- **target_degree** – float. Desired average degree to achieve after thresholding.
- **resolution** – int, optional (default=100). Number of threshold candidates between min and max to evaluate.

Returns

best_threshold – float. Threshold value that yields an average degree closest to target_degree.

BinarizeDFC(
time_series,
target_degree,
window,
lag,
tag='original',
)

Compute and binarize dynamic functional connectivity.

Parameters

- **time_series** – np.ndarray, shape (T, N). Input time series with T time points and N regions/nodes.
- **target_degree** – float. Desired average degree per snapshot.
- **window** – int. Length of sliding window.
- **lag** – int. Step size between windows.
- **tag** – str, optional. Binarization mode: *original* for per-snapshot thresholding, or *emp_static* for one static FC repeated across time.

Returns

tnet – np.ndarray, shape (n_snapshots, N, N). Binarized temporal adjacency matrices.

BinarizeDFCFromConnectivity(
fcs,
target_degree,
tag='original',
threshold=None,
dtype=<class 'int'>,
)

Binarize existing connectivity without recomputing sliding-window correlations.

Parameters

- **fcs** – np.ndarray, shape (T, N, N). Temporal connectivity matrices (correlations or z-score products).
- **target_degree** – float. Desired average degree.
- **tag** – str, optional (default='original'). Use 'emp_static' for static FC repeated across frames.
- **threshold** – float or None, optional (default=None). Precomputed temporal threshold; ignored for 'emp_static'.
- **dtype** – numpy dtype, optional (default=int). Output adjacency dtype; uint8 avoids large integer tensors.

Returns

tnet – np.ndarray, shape (T, N, N). Binary temporal adjacency with diagonal ones.

7.2 Preparation

ResolveAnalysisTag(*tag*)

Route a supported analysis tag to its signal and network processing stages.

Parameters
tag – str. Empirical, signal-surrogate, or network-null selection.

Returns

specification –

tuple. (phase_mode, binarization_tag, network_tag, seed_id).
phase_mode is None, 0, or 1. Stable seed IDs do not depend on tag order.

PrepareSignal(*tseries*, *tag*, *random_seed=None*)

Prepare a signal-based surrogate, or retain the original signals.

Parameters

- **tseries** – np.ndarray, shape (time, regions). Original subject signals.
- **tag** – str. Supported analysis tag.
- **random_seed** – int or None, optional (default=None). Signal-surrogate seed.

Returns
signals – np.ndarray. Original or phase-randomized signals. network_tag: str. Subsequent network-null selection.

PrepareConnectivity(
tseries,
tag='original',
window=20,
lag=1,
fc_method='original1',
random_seed=None,
)

Prepare the selected signals and compute Pearson or pointwise connectivity.

Parameters

- **tseries** – np.ndarray, shape (time, regions). Original subject signals.
- **tag** – str, optional (default='original'). Supported analysis selection.
- **window** – int, optional (default=20). Ignored for original2.
- **lag** – int, optional (default=1). Ignored for original2.
- **fc_method** – {'original1', 'original2'}, optional (default='original1'). FC method.
- **random_seed** – int or None, optional (default=None). Signal-surrogate seed.

Returns
fcs – np.ndarray, shape (frames, regions, regions). Selected connectivity.

PrepareTemporalNetwork(
fcs,
tag,
target_degree,
threshold=None,
seed=None,
dtype=<class 'numpy.uint8'>,
)

Binarize prepared connectivity and apply only the selected network-null step.

Parameters

- **fcs** – np.ndarray, shape (frames, regions, regions). Already prepared FC; for phase tags this must have been computed from the surrogate signals.
- **tag** – str. Supported analysis selection.
- **target_degree** – float. Target average degree.
- **threshold** – float or None, optional (default=None). Cached temporal threshold.
- **seed** – int or None, optional (default=None). Network-null seed.
- **dtype** – numpy dtype, optional (default=np.uint8). Output adjacency dtype.

Returns

tnet – np.ndarray. Binarized temporal network with waiting diagonals.

7.3 Surrogates

`PhaseRandSurrogates`(*TS*, *value=1*, *random_seed=None*)

Generate phase-randomized signals following dFCwalk's PhaseRand_surrogates.m.

Parameters

- **TS** – np.ndarray, shape (time, regions). Finite, real-valued input signals.
- **value** – {0, 1}, optional (default=1). Independent phases per region (0), or shared phases across regions (1).
- **random_seed** – int or None, optional (default=None). Seed for a local RNG.

Returns

TS_rnd – np.ndarray, shape (time, regions). Real-valued surrogate signals.

Notes

- Both modes preserve means and each region's Fourier magnitudes, hence its circular autocovariance. DC and even-length Nyquist terms are unchanged.
- Mode 1 also preserves cross-spectra and the sample covariance matrix.
- Mode 0 does not preserve interregional covariance in general.
- Seeded results are reproducible in Python, not seed-identical to MATLAB.

7.4 Distance and return

`SetDiagonals`(*tensor*, *value=1*)

Set the diagonals of all matrices in a temporal or static adjacency tensor.

Parameters

- **tensor** – np.ndarray, shape (T, N, N) or (N, N). Input adjacency tensor or matrix. If 2D, it is treated as a single snapshot.
- **value** – int or float, optional (default=1). Value to assign along the diagonal.

Returns

tensor_out – np.ndarray. The modified tensor with diagonals set to *value*.

ComputeAverageDegree(*adj_tensor*, *threshold=None*)

Compute the average and standard deviation of degrees over a temporal network.

Parameters

- **adj_tensor** – np.ndarray, shape (T, N, N). Temporal adjacency (binary or weighted). Assumed symmetric per snapshot.
- **threshold** – float or None, optional (default=None). If provided, an edge is A > threshold. If None, every nonzero entry is an edge.

Returns

mean_deg – float. Mean degree averaged over all nodes and time snapshots. std_deg: float. Standard deviation of degrees over all nodes and time snapshots.

Notes

- Diagonals are ignored without modifying the caller's array.
- Degree counts edges, not weighted strength. Directed inputs count outgoing edges.
- Process one frame at a time; no full copied/binarized temporal tensor.

SmartWalker(*tnet*, *hops_per_frame=1*, *latency_unit='frames'*)

Compute earliest-arrival temporal distances using a 'smart walker'. Propagate bit-packed reachability, recording first arrivals and stopping finished sources. Waiting makes previously reached targets stay reachable.

Parameters

- **tnet** – np.ndarray, shape (T, N, N). Temporal adjacency (binary), assumed symmetric per snapshot. Diagonals are forced to 1 internally (self-reachability).
- **hops_per_frame** – int, optional (default=1). Maximum propagation steps within each frame.
- **latency_unit** – {"frames", "frame_fraction", "substeps"}, optional (default="frames"). Unit of finite distances.

Returns

dmtx – np.ndarray, shape (N, N). Earliest arrival from i to j in the requested unit; np.nan on diagonal and np.inf for unreachable pairs.

RandomWalker(
tnet,
n_trials,
return_fpt_matrix=False,
seed=None,
summary_stat='mean',
hops_per_frame=1,
latency_unit='frames',
penalty=None,
)

Estimate temporal distances via compiled Monte Carlo random walks. For each trial and source node, simulate a walk over snapshots and record first-passage times.

Parameters

- **tnet** – np.ndarray, shape (T, N, N). Temporal adjacency (binary), assumed symmetric per snapshot. Diagonals are forced to 1 internally (self-reachability).
- **n_trials** – int. Number of Monte Carlo trials.
- **return_fpt_matrix** – bool, optional (default=False). If True, also return the per-trial FPT tensors.

- **seed** – int or None, optional (default=None). Seed for reproducibility.
- **summary_stat** – {"mean", "both"}, optional (default="mean"). Return five values or three adjacent mean/median pairs followed by two counts.
- **hops_per_frame** – int, optional (default=1). Maximum random-walk steps within each frame.
- **latency_unit** – {"frames", "frame_fraction", "substeps"}, optional (default="frames"). Unit of finite distances.
- **penalty** – float or None, optional (default=None). Unreachable-pair replacement; defaults to one frame beyond the horizon in the requested unit.

Returns

dmtx – np.ndarray, shape (N, N). Mean first passage time across trials, ignoring np.inf entries. np.nan on diagonal; np.inf if a node is never reached across all trials. mean_metrics: list[float]. Trial-averaged summary statistics (five by default, eight for "both"). all_fpt: np.ndarray, optional, shape (n_trials, N, N). Returned only if *return_fpt_matrix=True*.

Notes

Requires Numba. A fixed seed reproduces this compiled implementation;

MeanLatencyMatrixAnalysis(*dmx*, *penalty*, *summary_stat='mean'*)

Summarize a distance/first-passage matrix with five quantities. Given a matrix *dmx* where diagonal is NaN, finite off-diagonals are distances, and +inf denotes unreachable pairs, compute: 1) impermeability: Mean of distance normalized by source irrigation, averaged over sources. 2) latency: Mean of all finite distances. 3) resistance: Mean distance after replacing *+inf* with *penalty*. 4) inaccessibility: Count of *+inf* entries. 5) irrigation: Count of finite entries.

Parameters

- **dmx** – np.ndarray, shape (N, N). Distance or first-passage matrix with NaN on the diagonal and possibly +inf.
- **penalty** – float. Replacement value used to compute penalized mean distance.
- **summary_stat** – {"mean", "both"}, optional (default="mean"). For "both", interleave mean and median for the first three measures, then return each count once.

Returns

stats – list[float]. Five values by default, or eight values for "both".

ComputeSmartTemporalDistanceMeasures(
tnet,
random_seed=None,
summary_stat='mean',
hops_per_frame=1,
latency_unit='frames',
penalty=None,
)

Compute the smart-walker distance summary for a temporal network.

Parameters

- **tnet** – np.ndarray, shape (T, N, N). Temporal adjacency (binary), symmetric per snapshot.
- **random_seed** – int or None, optional (default=None). Seed for reproducibility (used by RandomWalker).
- **summary_stat** – {"mean", "both"}, optional (default="mean"). Select the five-value or eight-value mixed schema.
- **hops_per_frame** – int, optional (default=1). Maximum propagation steps within each frame.

- **latency_unit** – {“frames”, “frame_fraction”, “substeps”}, optional (default=”frames”). Unit of finite distances.
- **penalty** – float or None, optional (default=None). Unreachable-pair replacement in the requested unit.

Returns

metrics – list. Five values by default, or three mean/median pairs followed by two counts.

ComputeDrunkTemporalDistanceMeasures(
tnet,
n_reps=100,
random_seed=None,
summary_stat='mean',
hops_per_frame=1,
latency_unit='frames',
penalty=None,
)

Compute the drunk-walker distance summary for a temporal network.

Parameters

- **tnet** – np.ndarray, shape (T, N, N). Temporal adjacency (binary), symmetric per snapshot.
- **n_reps** – int, optional (default=100). Number of random-walk trials for the Monte Carlo estimator.
- **random_seed** – int or None, optional (default=None). Seed for reproducibility (used by RandomWalker).
- **summary_stat** – {“mean”, “both”}, optional (default=”mean”). Select the five-value or eight-value mixed schema.
- **hops_per_frame** – int, optional (default=1). Maximum random-walk steps within each frame.
- **latency_unit** – {“frames”, “frame_fraction”, “substeps”}, optional (default=”frames”). Unit of finite distances.
- **penalty** – float or None, optional (default=None). Unreachable-pair replacement in the requested unit.

Returns

metrics – list. Five values by default, or eight values averaged across random-walk trials.

ComputeReturnLatency(
adjacency_matrices,
max_latency=None,
return_full=False,
summary_stat=None,
hops_per_frame=1,
latency_unit='frames',
)

Compute node-wise return latency; one-hop searches use packed reachability.

Parameters

- **adjacency_matrices** – array, shape (T, N, N). Sequence of adjacency matrices (binary, 0/1), with waiting (diagonal ones).
- **max_latency** – int, optional. Maximum number of steps to search. If None, set to T.
- **return_full** – bool, optional. If True, return nodewise details and summary scalars.
- **summary_stat** – {None, “percentiles”, “mean”, “median”, “both”}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].

- **hops_per_frame** – int, optional (default=1). Maximum propagation steps within each frame.
- **latency_unit** – {"frames", "frame_fraction", "substeps"}, optional (default="frames"). Unit of return latency.

Returns

If *return_full* is False, *(summary_latency, mean_count_hits)*. If *return_full* is True, a dict with nodewise arrays plus global mean, std, median, and requested summary latency.

7.5 Dynamism and memory

`ComputeTemporalMutualInformation`(*adjacency_matrices*)

Compute average normalized mutual information between consecutive adjacency snapshots, optimized for symmetric (undirected) networks.

Parameters

- **adjacency_matrices** – array, shape (T, N, N).
- **Sequence of binary adjacency matrices** (*0/1*)

Returns

avg_mutual_information – float. Mean normalized mutual information between consecutive snapshots.

`ComputeDynamism`(*tnet*)

Summarize global temporal "dynamism" using entropy, transitions, and similarity. Given a binary temporal network *tnet* (T, N, N), this computes: 1) transition_probability, 2) global_entropy, 3) cosine_similarity, 4) net_fluidity, 5) mutual_information, and 6) mean_edge_count.

Notes

- Uses lower-triangular edges (i<j) for all edgewise computations.
- Entropy term gEnt excludes edges that are always 0 or always 1 to avoid log(0).
- Entropy normalization uses: maxH = -log(0.5) * [N(N-1)/2].
- Cosine similarity is computed on flattened lower-triangular edge vectors; rows with zero norm (empty graphs) are handled so that sim(t,t+1)=1 when both are empty, and rows involving an empty snapshot are set to 0 elsewhere to avoid spurious effects.

Parameters

tnet – np.ndarray, shape (T, N, N). Binary temporal adjacency (symmetric snapshots expected).

Returns

transition_probability – float. Estimated edge flip rate across consecutive snapshots. global_entropy: float. Normalized global entropy of edge activation probabilities. cosine_similarity: float. Mean cosine similarity between successive edge-state vectors. net_fluidity: float. Combined dynamism index *global_entropy * (1 - cosine_similarity)*. mutual_information: float. Average normalized mutual information across consecutive snapshots. mean_edge_count: float. Average undirected edge count per snapshot.

Notes

Requires *ComputeTemporalMutualInformation(tnet)* to be available in scope.

ComputeTemporalEdgeOverlap(*adjacency_matrices*)

Average Jaccard overlap of edge sets between consecutive snapshots. For symmetric binary matrices with ones on the diagonal. Only i<j edges are used.

Parameters
adjacency_matrices – np.ndarray, shape (T, N, N). Binary temporal adjacency.

Returns
avg_jaccard – float. Mean Jaccard similarity across consecutive pairs (t, t+1). NaN if T < 2 or all unions are empty.

ComputeEdgePersistenceRate(*adjacency_matrices*)

Fraction of active edges at time t that remain active at time t+1, averaged over t.

Parameters
adjacency_matrices – np.ndarray, shape (T, N, N). Binary temporal adjacency.

Returns
avg_persist – float. Mean persistence fraction over source frames t < T-1 with at least one edge. NaN if there are no eligible frames. Frames, not individual edges, receive equal weight.

ComputeNeighborhoodMemory(
adjacency_matrices,
lag=1,
summary_stat=None,
)

Compute neighborhood memory for each node via Jaccard similarity of neighbor sets. For each node i and snapshot t, compare its neighbors at t with its neighbors at t+lag. High memory indicates that a node tends to retain the same partners across time separated by the lag.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N). Sequence of binary adjacency matrices (0/1). Symmetric, diagonal ignored.
- **lag** – int, optional (default=1). Temporal lag between snapshots to compare (1 = consecutive snapshots).
- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].

Returns
summary – list[float] or float. Five percentiles by default, or the requested scalar summary.

Notes

- Node i's score is the mean Jaccard similarity across all valid (t, t+lag) pairs.
- If a node has no neighbors at both t and t+lag, that pair is skipped.
- Nodes with no valid pairs get memory = 0.

ComputeRetention(
adjacency_matrices,
summary_stat=None,
*,
delta=None,
)

Compute node-wise retention in a temporal network. The instantaneous score is the fraction of current neighbors encountered previously. With delta=None, all earlier frames are eligible. With positive integer delta, only frames max(0,t-delta) through t-1 are eligible. A node's score averages eligible nonempty frames.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N). Sequence of binary adjacency matrices (0/1). Symmetric, diagonal ignored.
- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].
- **delta** – positive int or None, optional (default=None). Lookback lifetime in adjacency frames; None retains unlimited history. delta=1 compares against the immediately preceding frame.

Returns

summary – list[float] or float. Five percentiles by default, or the requested scalar summary.

Notes

- The first nonempty neighbor observation is excluded.
- Empty current neighborhoods are skipped. After a node's first nonempty observation, a nonempty frame with no eligible recent partners contributes zero, even when all previous contacts have expired.
- Nodes with fewer than two nonempty observations have NaN scores. Self-connections are excluded. Current-frame contacts cannot count as their own history; each partner's lifetime resets when it is observed.
- delta counts adjacency frames, not FC window length, hop substeps or elapsed physical time. There is no wrapping at recording boundaries.
- delta=None uses unlimited history; a positive delta restricts the lookback lifetime. Both cached compiled modes take O(T*N^2) time and O(N) workspace beyond the input/output; finite windows do not require copying past frames.

ComputeLinkBurstiness(
adjacency_matrices,
*,
event_mode='activations',
)

Compute burstiness index B = (σ - μ) / (σ + μ) per link over time.

Parameters

- **adjacency_matrices** – array-like, shape (T, N, N). Sequence of binary (0/1) adjacency matrices for an undirected network. Diagonal is ignored if present.
- **event_mode** – {"activations", "onsets"}, optional (default="activations"). Count every active frame, or only the start of each uninterrupted active run.

Returns

link_burstiness – np.ndarray, shape (N, N). Burstiness per link; NaN where a link has fewer than 3 events in the selected mode. Diagonal is NaN.

Notes

- Uses population standard deviation of the observed inter-event gaps. Regular intervals give -1; positive values indicate variable gaps. A sustained link gives -1 in activation mode, but only one event (NaN) in onset mode. These definitions answer different questions.
- The uncorrected burstiness statistic is sample-size dependent; values based on only a few gaps should not be treated as reliable inference. Recording-boundary gaps are excluded; no periodic wrapping is assumed.
- Streaming compiled accumulation uses O(N^2) workspace, no copied T-by-N-by-N binary tensor or per-link event lists. Input is never modified.

7.6 Structure

ComputeSpatialClustering(
adjacency_matrices,
summary_stat=None,
)

Compute average spatial clustering coefficient per node across time. For each snapshot, the clustering coefficient of node i is defined as the fraction of realized links among i's neighbors out of the maximum possible. Final score per node is the time-average across snapshots.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N). Sequence of binary adjacency matrices (0/1). Symmetric; provide zero diagonals to exclude self-neighbors.
- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].

Returns

summary – list[float] or float. Five percentiles by default, or the requested scalar summary.

Notes

- Nodes with degree < 2 get clustering = 0 for that snapshot.
- This is the time-averaged version of the classic Watts–Strogatz clustering coefficient.
- Cached compiled kernels count links using packed neighbor intersections.

ComputeTemporalClustering(
adjacency_matrices,
delta=1,
summary_stat=None,
)

Compute temporal clustering coefficient for each node based on time-respecting triangles. A temporal triangle for node i is formed if: - i–j is active at time t1, - i–k is active at time t2 with t1 < t2 ≤ t1+delta, - and j–k is connected at some time t3 with t2 ≤ t3 ≤ t2+delta-1. Each temporal triplet is counted once even if its closing edge appears in multiple eligible frames.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N). Sequence of binary adjacency matrices (0/1). Symmetric; provide zero diagonals to exclude self-neighbors.
- **delta** – int, optional (default=1). Shared tolerance in frames. The second central connection may occur up to delta frames after the first; the closing edge is accepted within delta frames starting at the second connection's frame.

- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].

Returns

summary – list[float] or float. Five percentiles by default, or the requested scalar summary.

Notes

- Nodes with no valid triplets get clustering = 0.
- delta=1 uses consecutive snapshots, with closure in the second snapshot.
- Closing edges are accepted only once both central connections have been observed.
- Closing windows stop at the end of the series without wrapping. Eligible triplets with shorter remaining windows stay in the denominator.
- Packed closing-edge unions are computed once per frame; intersections avoid enumerating individual triplets.

ComputeSnapshotTransitivity(*adj_matrix*)

Global transitivity (clustering) for an undirected simple graph snapshot. Positive weights define edges; diagonal is ignored. Triangles are counted by sparse neighbor intersections rather than dense matrix cubing.

Parameters

adj_matrix – np.ndarray or scipy sparse matrix, shape (N, N). Nonnegative symmetric adjacency; positive entries define edges, diagonal ignored.

Returns

transitivity – float. 3 * (#triangles) / (#connected triplets). Returns 0 if no triplets exist.

ComputeTemporalTransitivity(*adjacency_matrices*)

Average snapshot transitivity (global clustering) over time.

Parameters

adjacency_matrices – np.ndarray, shape (T, N, N). Binary temporal adjacency.

Returns

avg_transitivity – float. Mean transitivity across snapshots (ignores NaNs).

ComputeSnapshotParticipationCoefficient(
adj_matrix,
labels,
eps=1e-12,
)

Participation coefficient (network segregation/integration per node). For node i: P_i = 1 - sum_c (k_i^c / k_i)^2, where k_i^c is strength/degree of i to community c. Returns NaN for isolated nodes (k_i = 0).

Parameters

- **adj_matrix** – np.ndarray, shape (N, N). Binary or weighted symmetric adjacency (diag ignored).
- **labels** – array-like, shape (N,). Community labels per node.
- **eps** – float, optional (default=1e-12). Numerical guard to avoid division-by-zero issues.

Returns

P – np.ndarray, shape (N,). Participation coefficient per node (NaN for isolated nodes).

ComputeTemporalParticipationCoefficient(
adjacency_matrices,
labels_per_snapshot,
)

Average participation coefficient over time.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N).
- **labels_per_snapshot** – np.ndarray, shape (T, N) or (N,). A fixed partition may be shared across frames.

Returns

avg_participation – float. Mean of nodewise participation coefficients averaged across time (ignores NaNs for isolated nodes). P_all: np.ndarray, shape (T, N). Per-snapshot nodewise participation coefficients.

ComputePartnerStability(
adjacency_matrices,
summary_stat=None,
***,
normalization='duration',
)

Compute partner stability for each node, based on recurrence of connections over time. Stability is higher when neighbor interactions recur more frequently, i.e., when the mean temporal gap between repeated connections is smaller.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N). Sequence of binary adjacency matrices (0/1). Symmetric, diagonal ignored.
- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].
- **normalization** – {"duration", "gap"}, optional (default="duration"). 1-mean_gap/T score, or reciprocal mean gap in frames.

Returns

summary – list[float] or float. Five percentiles by default, or the requested scalar summary.

Notes

- Node i's score is computed as 1 - mean_gap/T, where mean_gap is the average interval (in snapshots) between consecutive interactions with the same neighbor.
- Nodes with no recurring neighbors get stability = 0.
- This is a horizon-normalized recurrence-gap index, not adjacent-frame edge survival. Scores from different recording lengths are not directly comparable. All recurrence intervals, not partners, have equal weight.
- normalization='gap' gives 1/mean_gap, avoiding normalization by recording duration. It is a recurrence rate per frame, not a probability of next-frame edge survival; frame duration still matters.

ComputePartnerDiversity(
adjacency_matrices,
window_size=20,
summary_stat=None,
***,
normalization='observed',
)

Compute partner diversity for each node using normalized entropy across temporal windows. The default measures evenness of partners' window coverage, not the number of partners or temporal ordering. Equal coverage by two or many partners both scores 1. Use normalization='available' to also reflect partner breadth.

Parameters

- **adjacency_matrices** – np.ndarray, shape (T, N, N). Sequence of binary adjacency matrices (0/1). Symmetric, diagonal ignored.
- **window_size** – int, optional (default=20). Temporal window size for grouping interactions.
- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].
- **normalization** – {"observed", "available"}, optional (default="observed"). Normalize entropy by log(observed partners) or log(N-1).

Returns

summary – list[float] or float. Five percentiles by default, or the requested scalar summary.

Notes

- For each neighbor j of node i, we collect the set of windows where i–j connections occurred. Temporal span per neighbor is normalized by total windows.
- Diversity is then computed as the entropy of these spans, normalized by log(# of distinct partners).
- Nodes with zero or one partner get diversity = 0.
- A final partial window is included; windows longer than the recording form one window. Window coverage ignores repeated events within a window.

```
ComputeNodePersistence(
    tnet,
    confidence=0.95,
    summary_stat=None,
    *,
    null_probability=None,
    return_details=False,
)
```

Compute occupancy among ever-connected partners and an iid-null exceedance count. Persistence is the mean active-frame fraction among a node's ever-connected partners, excluding self-connections. This is occupancy, not edge survival.

Parameters

- **tnet** – np.ndarray, shape (T, N, N). Temporal adjacency matrices (binary or weighted). Symmetric assumed.
- **confidence** – float, optional (default=0.95). One-sided confidence level; compare conditional upper-tail probabilities with 1-confidence.
- **summary_stat** – {None, "percentiles", "mean", "median", "both"}, optional (default=None). Return five percentiles by default, one scalar, or [mean, median].
- **null_probability** – float or None, optional (default=None). Independent Bernoulli edge probability. None estimates it from off-diagonal density.
- **return_details** – bool, optional (default=False). Append a dictionary with node scores, conditional tail probabilities and the null probability.

Returns

summary – list[float] or float. Five percentiles by default, or the requested scalar summary. n_persistent_nodes: int. Number of nodes with conditional upper-tail probability < 1-confidence. No multiple-testing correction is applied. details: dict, optional. Present only when return_details=True.

Notes

- Weighted inputs are binarized (>0 → 1).
- Node i's score is the average persistence probability across all neighbors that ever connected to i.
- Conditional on K ever-active partners, their activation counts follow K independent Binomial(T,p) distributions conditioned on positive counts. Their sum's upper tail is computed by FFT convolution (not a normal approximation).
- The null assumes homogeneous independent links and independent frames. Overlapping FC windows generally violate this assumption. With estimated p, these are plug-in model tail probabilities, NOT calibrated significance claims. Use independently specified p or appropriate surrogates for inference.
- Isolated nodes have NaN scores and are never counted. Diagonal ignored.

7.7 Modularity

EnsureCsr(*A*)

Copy an adjacency matrix into canonical CSR format with zero diagonal.

Parameters

A – array-like or scipy.sparse.spmatrix, shape (N, N). Binary or nonnegative weighted adjacency; directed inputs are also accepted here.

Returns

A_csr – scipy.sparse.csr_matrix. Float adjacency with zero diagonal, duplicate entries summed and explicit zeros removed.

Notes

- Never modifies the caller's dense or sparse input.
- Non-finite or negative off-diagonal weights are rejected; signed modularity requires a separate null model.

CoarseGrain(*A*, *comm*, *C*)

Sum current graph weights between communities into a coarse graph.

Parameters

- **A** – scipy.sparse.spmatrix, shape (N, N). Current-level adjacency, including any coarse self-loops.
- **comm** – array-like, shape (N,). Integer community identifiers in 0..C-1.
- **C** – int. Number of communities.

Returns

A_coarse – scipy.sparse.csr_matrix, shape (C, C). Canonical coarse graph, retaining self-loops and total weight.

Notes

- COO coordinates are paired directly with their weights, including explicit-zero or duplicate input entries.

DetectCommunitiesLouvain(
A,
gamma=1.0,
max_passes=100,
max_levels=10,
tol=1e-07,
seed=None,
***,
n_restarts=1,
)

Detect undirected weighted communities with sparse multilevel Louvain.

Parameters

- **A** – array-like or scipy.sparse.spmatrix, shape (N, N). Symmetric binary or nonnegative weighted adjacency; original diagonal ignored.
- **gamma** – float, optional (default=1.0). Nonnegative resolution parameter; larger values favor smaller communities.
- **max_passes** – int, optional (default=100). Maximum node sweeps at each level.
- **max_levels** – int, optional (default=10). Maximum aggregation levels per restart.
- **tol** – float, optional (default=1e-7). Minimum modularity improvement to continue to another level.
- **seed** – int or None, optional (default=None). Seed controlling node visitation order.
- **n_restarts** – int, keyword-only, optional (default=1). Independent visitation sequences; return the highest-Q partition found.

Returns

labels – np.ndarray, shape (N,), dtype=int. Community assignments of original nodes. Q: float. Modularity of the returned partition on the original zero-diagonal graph.

Notes

- Compiled sparse node sweeps avoid Python per-edge dictionaries; coarse self-loops are retained but excluded from move-edge totals.
- Disconnected detected communities are split into connected components.
- Louvain remains a heuristic with a modularity resolution limit; restarts do not guarantee a global optimum or constitute the Leiden algorithm.
- A fixed seed gives reproducible results in this implementation.
- Reference: Blondel et al. (2008), https://arxiv.org/abs/0803.0476.

SnapshotModularity(*A*, *labels*, *gamma=1.0*, *directed=False*)

Score a supplied snapshot partition using undirected or directed modularity.

Parameters

- **A** – array-like or scipy.sparse.spmatrix, shape (N, N). Binary or nonnegative weighted adjacency; diagonal ignored in both modes.
- **labels** – array-like, shape (N,). Community identifiers per node.
- **gamma** – float, optional (default=1.0). Finite nonnegative resolution parameter.

- **directed** – bool, optional (default=False). Use out-strength times in-strength in the null model instead of the undirected strength product.

Returns

Q – float. Snapshot modularity; zero for an edgeless graph.

Notes

- Does not modify inputs or densify sparse graphs.
- Negative weights need signed modularity and are rejected, not silently discarded.

ComputeSnapshotModularity(
adj_matrix,
labels=None,
***,
gamma=1.0,
directed=False,
detect_method='louvain',
detect_kwargs=None,
)

Score a snapshot partition, optionally detecting communities first.

Parameters

- **adj_matrix** – array-like or scipy.sparse.spmatrix, shape (N, N). Binary or nonnegative weighted adjacency; diagonal ignored.
- **labels** – None or array-like, shape (N,), optional (default=None). Supplied partition or automatic undirected detection.
- **gamma** – float, optional (default=1.0). Nonnegative modularity resolution parameter.
- **directed** – bool, optional (default=False). Directed scoring requires supplied labels.
- **detect_method** – {"louvain"}, optional (default="louvain"). Automatic community detection method.
- **detect_kwargs** – dict or None, optional (default=None). Detector options such as seed, max_passes or n_restarts.

Returns

Q – float. Snapshot modularity. used_labels: np.ndarray, shape (N,). Detected integer labels or an unmodified copy of supplied identifiers.

ComputeTemporalModularity(
adjacency_matrices,
labels_per_snapshot=None,
***,
gamma=1.0,
directed=False,
detect_method='louvain',
detect_kwargs=None,
)

Compute the arithmetic mean of independent snapshot modularities.

Parameters

- **adjacency_matrices** – np.ndarray or list, shape (T, N, N) or list of (N, N). Binary or nonnegative weighted snapshots, dense or sparse.
- **labels_per_snapshot** – None, (N,), or (T, N), optional (default=None). Detect independently per frame, reuse one supplied partition, or score frame-specific partitions.
- **gamma** – float, optional (default=1.0). Nonnegative modularity resolution parameter.

- **directed** – bool, optional (default=False). Directed scoring requires supplied partitions.
- **detect_method** – {"louvain"}, optional (default="louvain"). Automatic detection method.
- **detect_kwargs** – dict or None, optional (default=None). Detector options, e.g. {"seed": 42, "n_restarts": 5}.

Returns

avg_Q – float. Mean snapshot modularity; NaN for an empty sequence. Qs: np.ndarray, shape (T,). Snapshot modularity values. out_labels_per_snapshot: np.ndarray, shape (T, N). Detected or supplied labels used for each snapshot.

Notes

- This is not multilayer modularity: there is no inter-frame coupling or community-identity matching across snapshots.
- Edgeless frames contribute zero to the mean. Detection and scoring share one path, avoiding redundant per-frame scoring.
- The same explicitly supplied seed is forwarded to each snapshot for reproducibility.

7.8 Network generators

GenerateSymmetricRandomNetwork(*n_nodes*, *n_edges*)

Build a symmetric random adjacency with a full diagonal.

Parameters

- **n_nodes** – Number of nodes.
- **n_edges** – Number of undirected off-diagonal edges.

Returns

Symmetric *(n_nodes, n_nodes)* adjacency with diagonal ones.

GenerateSymmetricSmallWorldNetwork(
n_nodes,
n_edges,
rewire_prob=0.02,
seed=None,
)

Build a symmetric small-world-style adjacency with diagonal ones.

Parameters

- **n_nodes** – Number of nodes.
- **n_edges** – Number of undirected off-diagonal edges.
- **rewire_prob** – Fraction of initially local edges to rewire.
- **seed** – Optional RNG seed.

Returns

Symmetric *(n_nodes, n_nodes)* adjacency with diagonal ones.

GenerateSymmetricScaleFreeNetwork(
n_nodes,
n_edges,
gamma=2.5,
)

Build a symmetric power-law scale-free adjacency with diagonal ones.

Parameters

- **n_nodes** – Number of nodes.
- **n_edges** – Number of undirected off-diagonal edges.
- **gamma** – Power-law exponent used for node preference weights.

Returns

Symmetric *(n_nodes, n_nodes)* adjacency with diagonal ones.

GenerateSymmetricNetwork(*n_nodes*, *n_edges*, *kind*, *seed=None*)

Dispatch to an ER, small-world, or scale-free generator.

Parameters

- **n_nodes** – Number of nodes.
- **n_edges** – Number of undirected off-diagonal edges.
- **kind** – One of *er*, *sw*, or *sf*.
- **seed** – Optional RNG seed.

Returns

Symmetric *(n_nodes, n_nodes)* adjacency with diagonal ones.

GenerateStaticTemporalNetwork(*tnet_init*, *kind*, *seed=None*)

Repeat one static snapshot across all time points.

Parameters

- **tnet_init** – Reference temporal network of shape *(T, N, N)*.
- **kind** – One of *er*, *sw*, or *sf*.
- **seed** – Optional RNG seed.

Returns

Temporal network of shape *(T, N, N)*.

GenerateTemporalLinkCounts(
target_mean,
target_sigma,
n_snapshots,
max_attempts=2000,
)

Match per-snapshot link counts to a target mean and spread.

Parameters

- **target_mean** – Desired mean link count.
- **target_sigma** – Desired standard deviation.
- **n_snapshots** – Number of snapshots.
- **max_attempts** – Maximum redistribution attempts.

Returns

Integer array of length *n_snapshots*.

GenerateTemporalNetworkWithDensityVariation(
tnet0,
variation_type='uniform',
)

Regenerate a temporal network with uniform edge counts across time.

Parameters

- **tnet0** – Reference temporal network of shape *(T, N, N)*.

- **variation_type** – Must be *uniform*.

Returns
Temporal network of shape *(T, N, N)*.

GenerateTemporalNetworkByLinkActivation(
tnet_init,
seed=None,
)

Shuffle edge activation times while preserving per-edge totals.

Parameters

- **tnet_init** – Binary temporal network of shape *(T, N, N)*.
- **seed** – Optional RNG seed.

Returns
Temporal network of shape *(T, N, N)*.

GenerateTemporalNetworkFromEdgeTemplate(
otnet,
tag,
p_rewire=0.02,
seed=None,
alpha_rank=1.0,
exclude_never_active=True,
eta_readd=1.0,
)

Build a temporal network from an empirical or random ranked edge template.

Parameters

- **otnet** – Reference temporal network of shape *(T, N, N)*.
- **tag** – *eActTemplate* or *eRndTemplate*.
- **p_rewire** – Fraction of template edges rewired per frame.
- **seed** – Optional RNG seed.
- **alpha_rank** – Rank-decay exponent.
- **exclude_never_active** – If *True*, *eActTemplate* ignores never-active edges.
- **eta_readd** – Weight multiplier for re-adding removed edges.

Returns
Tuple *(tnet_out, n_final_core_nodes)*.

TrimIsolatedNodes(*tnet*)

Remove nodes that never connect to any other node across time.

GenerateNullModel(*tnet*, *tag*, *scale=1*, *seed=None*, ***kwargs*)

Generate a temporal null model from *tnet*.

Supported tags:
original, *emp_static*, *er_static*, *sw_static*, *sf_static*, *time*, *eAct*, *uniform*, *eActTemplate*, *eRndTemplate*, *erand_er*, *erand_sws*, *erand_sfs*, *erand_swr*, *erand_sfr*.

Parameters

- **tnet** – Input temporal network of shape *(T, N, N)*.
- **tag** – Null-model identifier.
- **scale** – Reserved argument; does not affect generation.
- **seed** – Optional RNG seed.

- ****kwargs** – Extra options forwarded to template-based generators.

Returns

Tuple *(tnet_out, n_final_core_nodes)*.

generateRandomTemporalNetwork (*t*, *n*, *pconn*)

Generate a symmetric random temporal network. This function creates a temporal network represented as a 3D NumPy array of shape (t, n, n), where each n x n slice along the time axis is a symmetric adjacency matrix representing the network at a given time step. Edges between distinct node pairs are included independently with probability *pconn*. All diagonal entries (self-loops) are set to 1.

Parameters

- **t** (*int*) – Duration of the temporal network (number of time steps).
- **n** (*int*) – Number of nodes in the network.
- **pconn** (*float*) – Probability of connection between distinct node pairs.

Returns

np.ndarray – A temporal network of shape (t, n, n).

PartiallyRandomizeMatrix (
adj_matrix,
rewire_prob,
preserve_first_snapshot=False,
seed=None,
)

Partially randomize a symmetric connectivity matrix while preserving self-loops.

GenerateRandomizedTemporalNetwork (
initial_matrix,
n_snapshots,
rewire_prob,
seed=None,
)

Generate a randomized temporal network from a static connectivity matrix. At each snapshot, the base adjacency matrix is partially randomized by rewiring a fraction of its edges while preserving the diagonal. The first snapshot can be preserved exactly.

The toolbox is distributed under the MIT license. Docstrings specify accepted inputs; this guide adds workflow and interpretation advice.